Recurrent Neural Network (RNN)

Introduction to Recurrent Neural Network, and LSTM

IEOR 4574: Forecasting: *A Real-World Application*

Syed Waseem Haider, Ph.D.

Dr.Waseem.Haider@gmail.com

April 16, 2026

Introduction

- A recurrent neural network (RNN) is theoretically Turing complete. A Turing complete language or model can simulate an arbitrary Turing Machine.
- A Turing machine is a mathematical computation model that reads and writes on an infinite tape (the memory). Turing machines also have a state transition table listing the computation's rules.
- Turing machine processes an infinite sequence of symbols. However, the lexicon of the symbols is finite.
- For example, well-known programming languages are Turing complete. It's a computational model and not a computation problem.
- A time series could be a sequence of symbols such as alphabets from a spoken language, the corpus of spoken words, or numbers from $\mathbb{R}$.
- RNN predicts the next number or word in the sequence. It could also predict a sub-sequence.

Introduction

- Some examples of RNN include Long Short Term Memory (LSTM) which addresses some of the shortcomings of theoretical RNNs.
- Feed-Forward Neural Networks (FFNN) are those in which information flows in one direction only. There are no feedback loops.
- FFNN is a memory-less system. Once the inputs are processed, they are forgotten.
- Basically, in FFNN, the weight matrix is multiplied by the inputs to produce outputs.
- On the contrary, RNN is a system with a memory. It retains what was processed in the last step through a feedback loop.
- Usually, in time series data, the successive data points have some kind of correlation.
- Autocorrelations in the time series were discussed in the Forecasting Introduction slides.

RNN

Basic Architecture

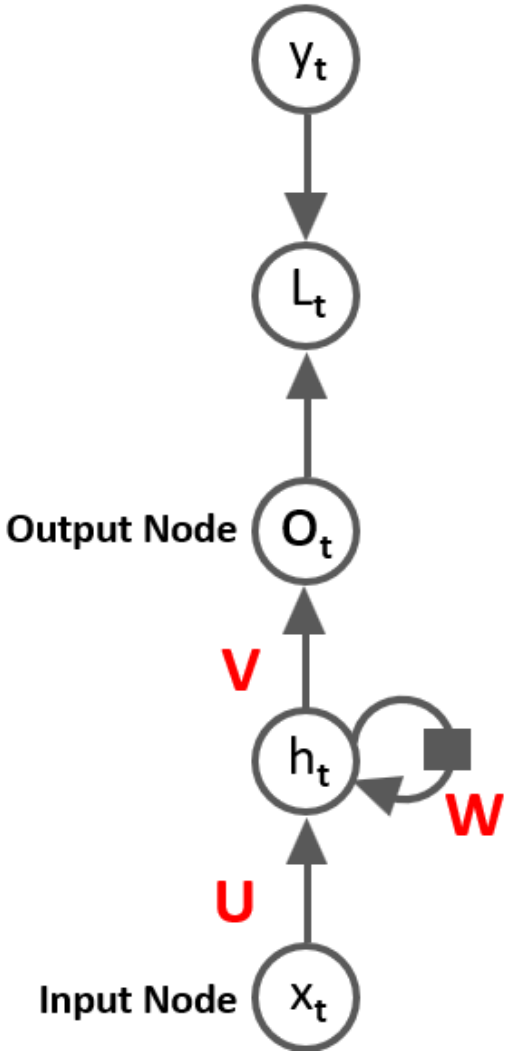


Figure: A single hidden node RNN.

RNN

Unrolled in Time

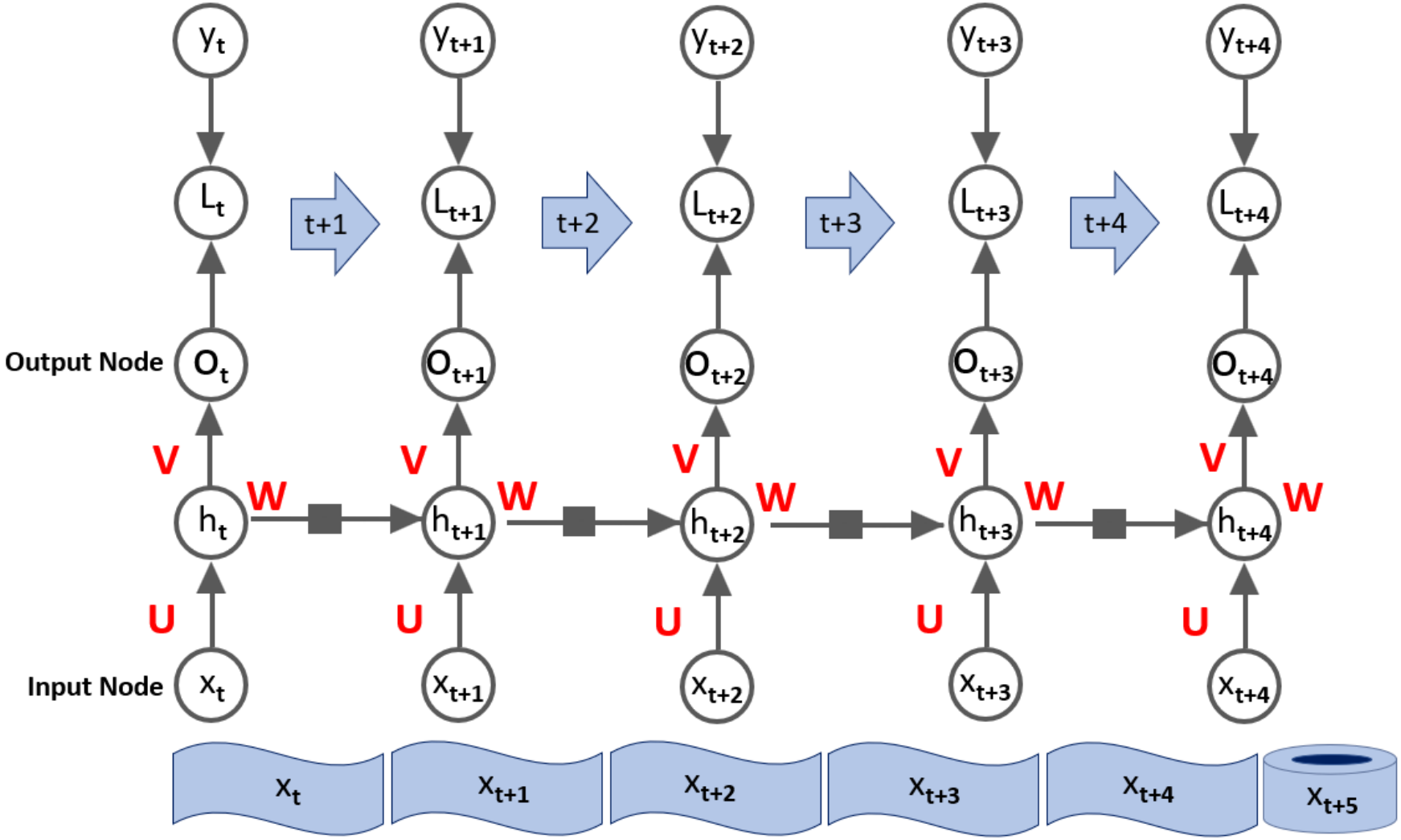


Figure: A time series data is unrolled in time with RNN being also unpacked.

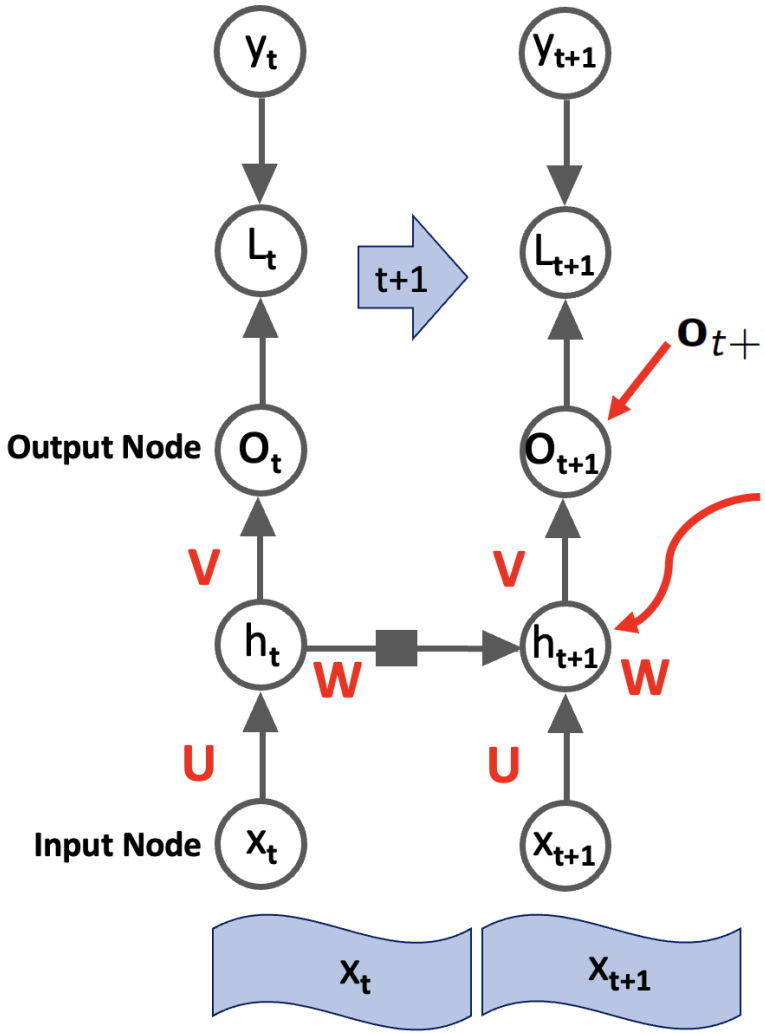
RNN

Basic Architecture

- $\{\mathbf{x}_t\}$ is the input time series, $\{\mathbf{y}_t\}$ is the actuals time series, and $\{\mathbf{o}_t\}$ is the unnormalized log probabilities time series.
- $\{\mathbf{h}_t\}$ is the hidden node value in time, which is also a time series. There is an activation function on the hidden node. In this case, it is $tanh(\cdot)$.
- Finally, $\{\mathbf{L}_t\}$ is the loss or error value in time, yet another time series.
- $\mathbf{U}$ is the input weight matrix, $\mathbf{V}$ is the output weight matrix, and $\mathbf{W}$ is the loopback (or recursive) weight matrix.
- $\{\mathbf{x}_t\}$ is the input tape that is unrolled in time. As $\{\mathbf{x}_t\}$ unrolls so does RNN unpack in time?
- $\mathbf{b}$ and $\mathbf{c}$ are the biases parameters.
- $\mathbf{a}_{t+1} = \mathbf{b} + \mathbf{W}\ \mathbf{h}_t + \mathbf{U}\ \mathbf{x}_{t+1}$
- $\mathbf{h}_{t+1} = tanh\left(\mathbf{a}_{t+1}\right) = tanh\left(\mathbf{b} + \mathbf{W}\ \mathbf{h}_t + \mathbf{U}\ \mathbf{x}_{t+1}\right)$.
- $\mathbf{o}_{t+1} = \mathbf{c} + \mathbf{V}\ \mathbf{h}_{t+1}$.

RNN

Basic Architecture



$$\mathbf{o}_{t+1} = \mathbf{c} + \mathbf{V}\ \mathbf{h}_{t+1}$$

$$\mathbf{a}_{t+1} = \mathbf{b} + \mathbf{W}\ \mathbf{h}_t + \mathbf{U}\ \mathbf{x}_{t+1}$$

$$\mathbf{h}_{t+1} = tanh\left(\mathbf{a}_{t+1}\right) = tanh\left(\mathbf{b} + \mathbf{W}\ \mathbf{h}_t + \mathbf{U}\ \mathbf{x}_{t+1}\right)$$

Figure: RNN Computations between x_t and x_{t+1}.

RNN

Basic Architecture

- When running the RNN, the matrices **U**, **V**, and **W** remain fixed in time.
- **U**, **V**, and **W** only change when RNN is being trained using the backpropagation.
- The prediction is the $\hat{\mathbf{y}}_{t+1} = softmax\left(\mathbf{o}_{t+1}\right)$.
- Softmax activation function $\Phi(v)_i = \frac{exp(v_i)}{\sum_{j=1}^{k} exp(v_j)} \quad \forall i \epsilon \{1, \ldots, k\}$, where $\Phi(\cdot)_i \ \epsilon \ \{0, 1\}$.
- Intuitively, $\Phi(v)_i = 1$ if it's i-th class. Conversely, $\Phi(v)_j = 0, \forall j \neq i$.
- Softmax activation function is used for multinomial distribution such as the next word from the corpus given the sequence of words observed so far, or the next alphabet from a vocabulary.
- The total Loss function is defined over the entire input and output sequence $L\left(\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}, \{\mathbf{y}_1, \ldots, \mathbf{y}_N\}\right)$.

RNN

Basic Architecture

- The loss function on each time step, $L_t = log\ P_{model}\left(\mathbf{y}_t \mid \{\mathbf{x}_1, \ldots, \mathbf{x}_t\}\right)$, is the log of the probability that the model predicts $\mathbf{y}_t$ after observing the data $\{\mathbf{x}_1, \ldots, \mathbf{x}_t\}$.
- Loss function L is defined over the joint probability distribution of the input sequence and the output sequence.
- The total loss function is the sum of loss on each time step. $L\left(\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}, \{\mathbf{y}_1, \ldots, \mathbf{y}_N\}\right) = \sum_t L_t = -\sum_t log\ P_{model}\left(\mathbf{y}_t \mid \{\mathbf{x}_1, \ldots, \mathbf{x}_t\}\right)$.
- For each node N in time the gradient is computed recursively starting with node immediately preceeding the final loss $\frac{\partial L}{\partial L_t} = 1$.
- Given the input so far $\{\mathbf{x}_i\}$, the loss is negative log-likelihood.

RNN

Back Propagation Through Time (BPTT)

- Note that the system is causal hence the probability of any event depending on future observations or inputs is zero. $P_{model}\left(\mathbf{y}_t \mid \{\mathbf{x}_1, \ldots, \mathbf{x}_{t+\tau}\}\right) = 0$ for $\tau > 0$.
- The log probability model automatically lands itself naturally into the Softmax activation function as discussed earlier.
- The training time is "big O", $O(N)$. Because of the recursive nature of the functions, first, all-time steps are sequentially processed in the forward direction and the result in each time step is stored.
- Then, the weight matrices and bias parameters are computed through the backpropagation method discussed earlier.
- The time it takes to do all of these steps is called BPTT, which cannot be parallelized because of the recursive nature of the architecture.
- Overall, RNN is very powerful but expensive to train.

RNN

Back Propagation Through Time (BPTT) with Time Sliced

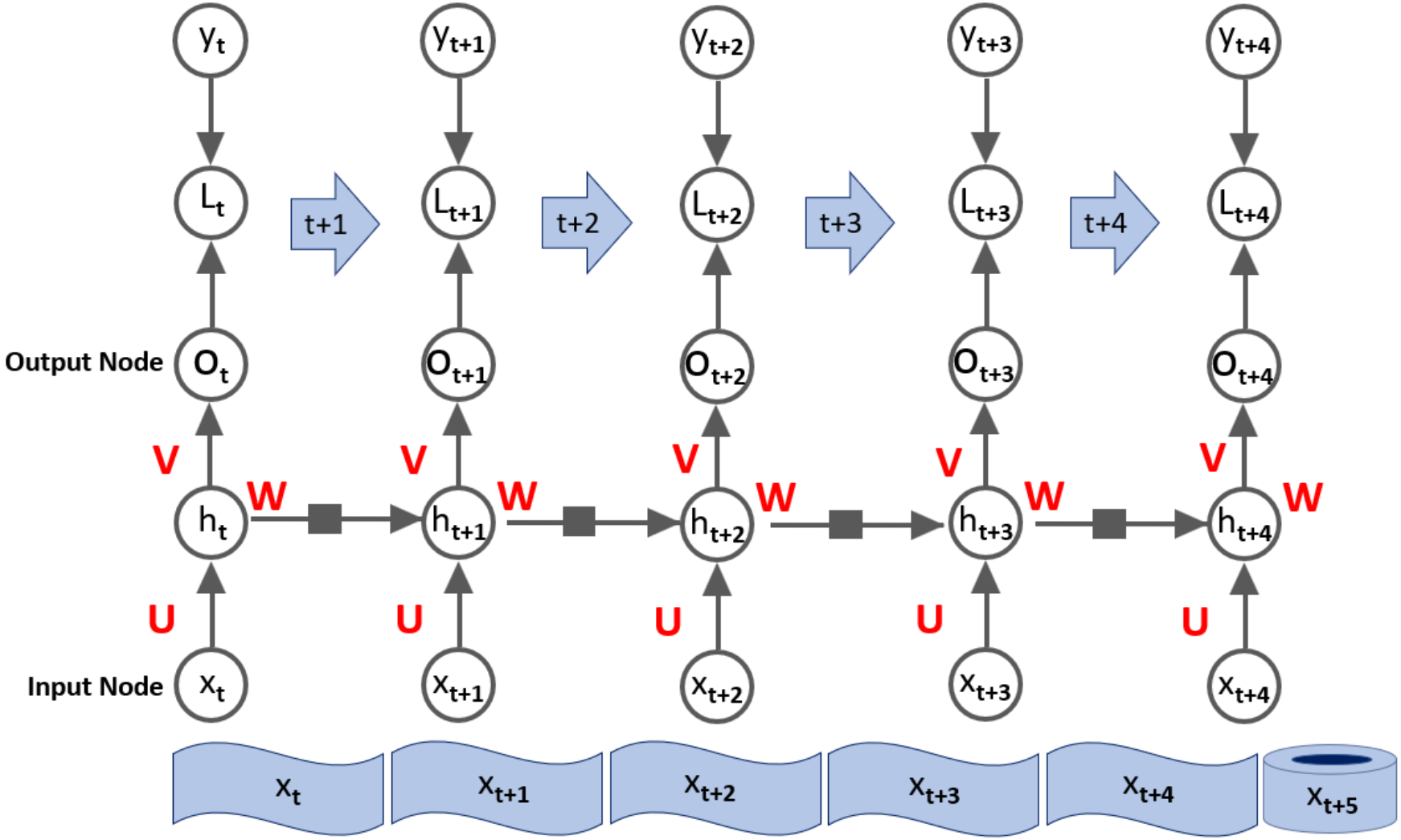


Figure: RNN is decomposed into t time steps.

RNN

Back Propagation Through Time (BPTT) with Time Sliced

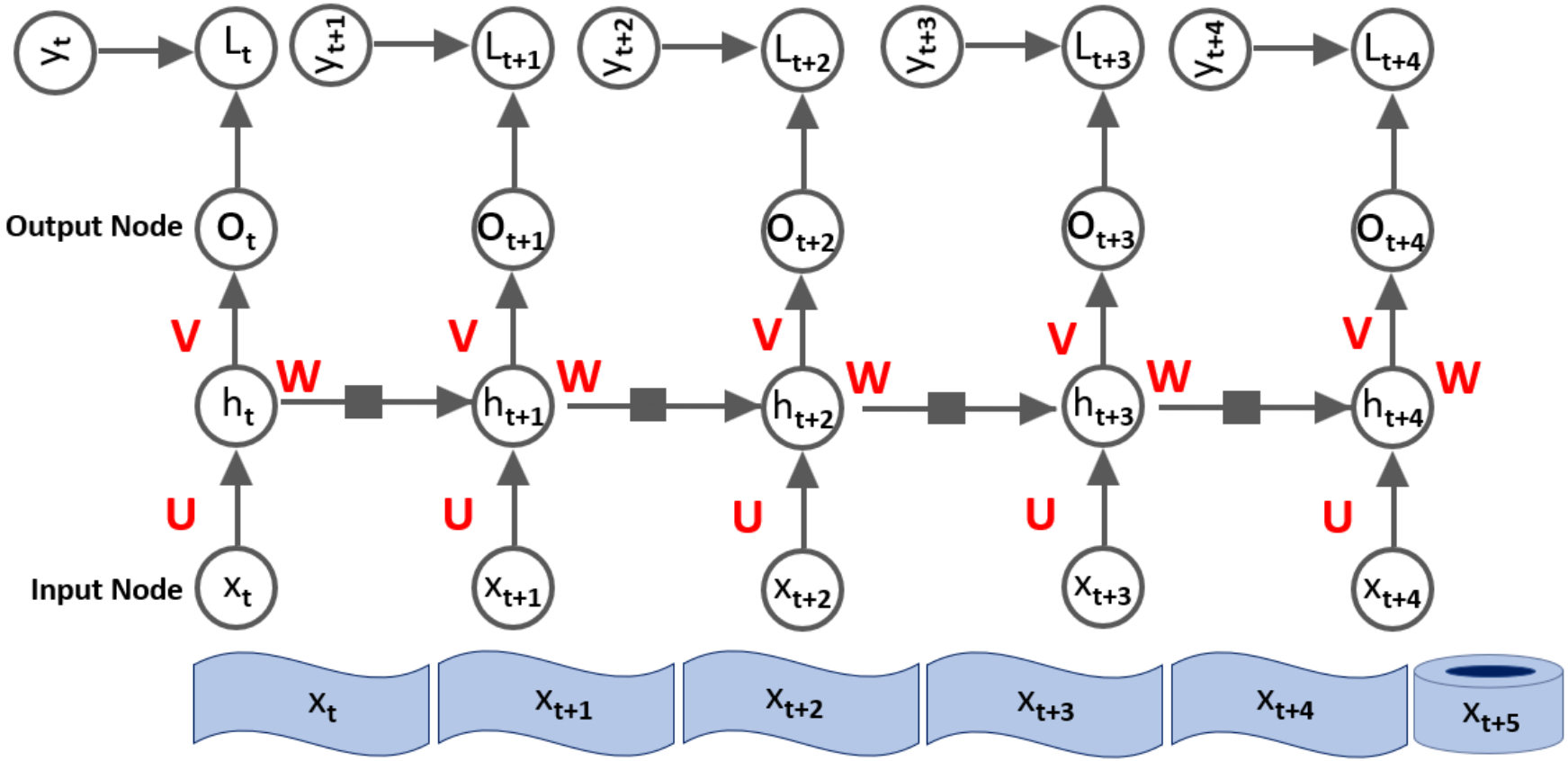


Figure: RNN is decomposed into t time steps.

RNN

Back Propagation Through Time (BPTT) with Time Sliced

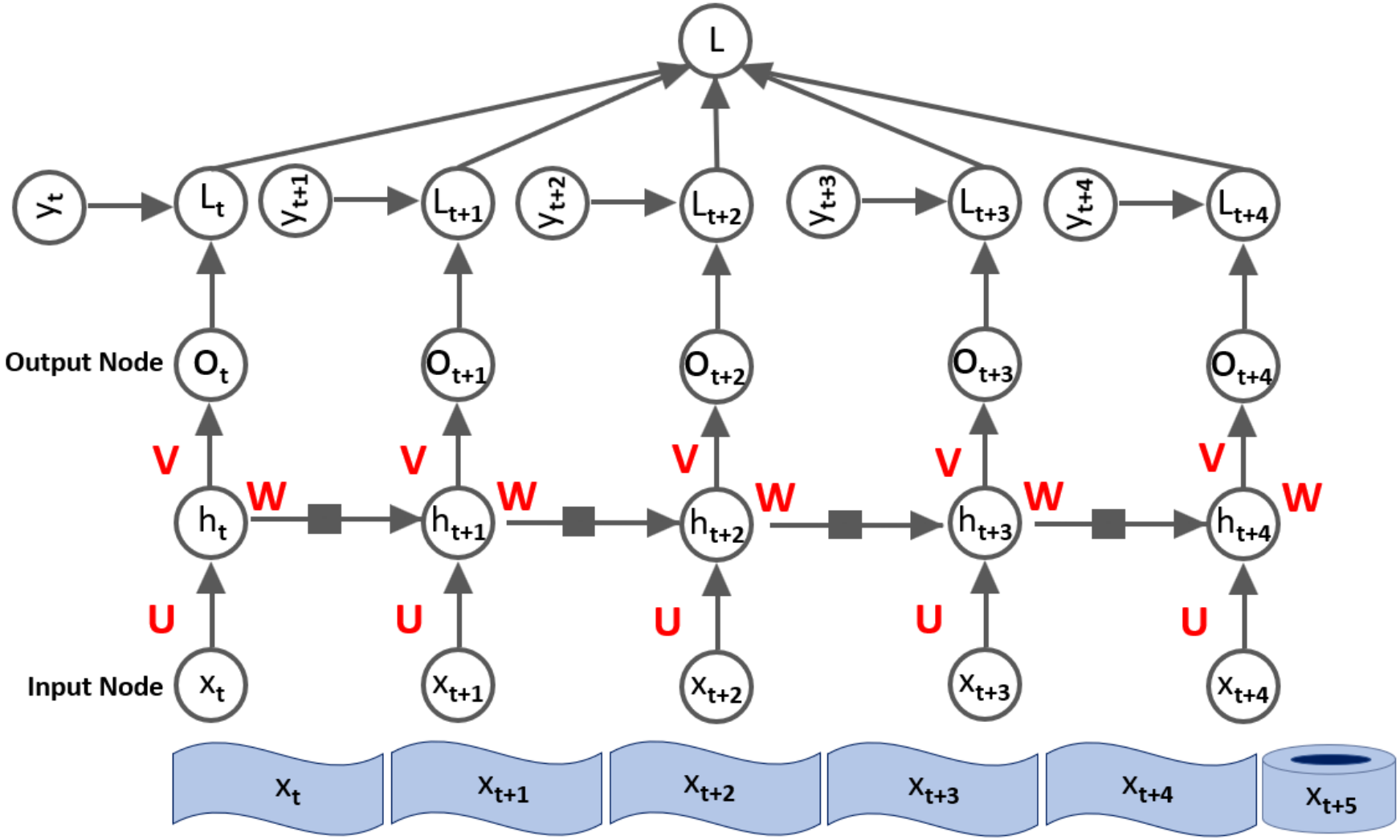


Figure: RNN is decomposed into t time steps.

RNN

Time Sliced

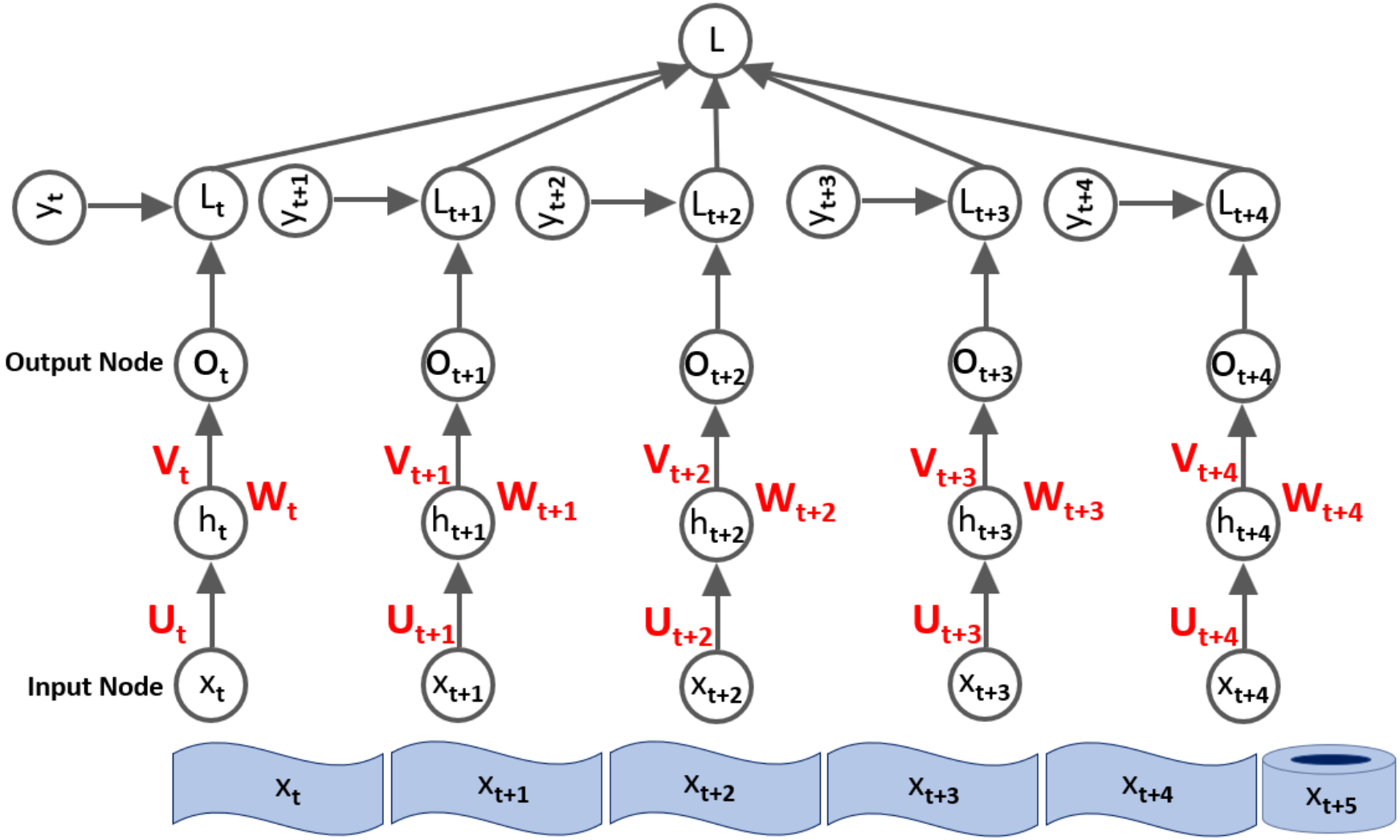


Figure: RNN is decomposed into t time steps.

RNN

Back Propagation Through Time (BPTT) with Time Sliced

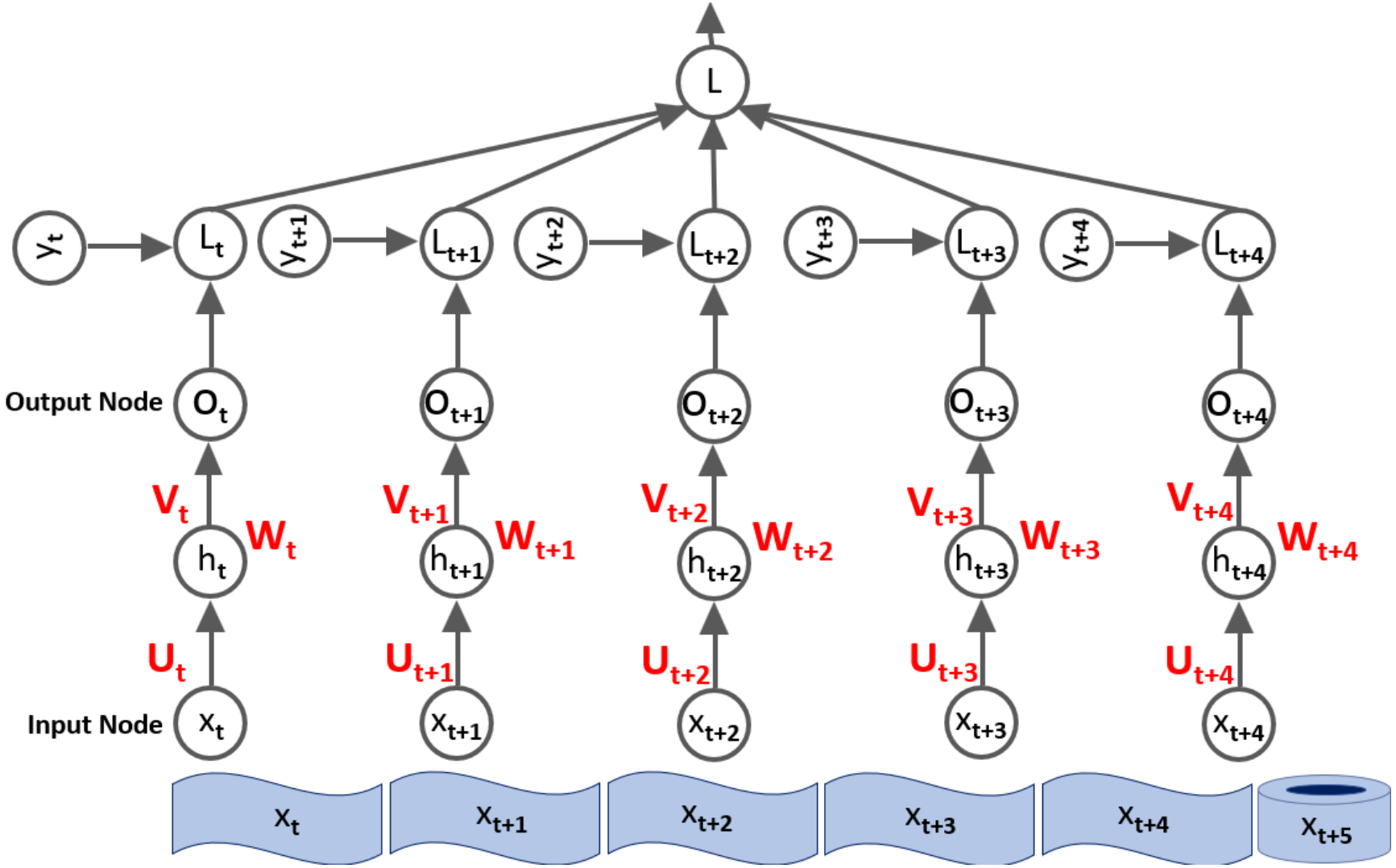


Figure: RNN is decomposed into t time steps.

RNN

Back Propagation Through Time (BPTT)

- $\mathbf{1}_{i,y_t}$ is the output vector which is 0 everywhere except at the actual output value. When $\hat{y}_{i,t} - \mathbf{1}_{i,y_t} < 0$ there is an error, and when $\hat{y}_{i,t} - \mathbf{1}_{i,y_t} = 0$ it's an accurate prediction.
- The gardient $(\nabla_{\mathbf{o_t}} L)_i = \frac{\partial L}{\partial o_{i,t}} = \frac{\partial L}{\partial L_t} \frac{\partial L_t}{\partial o_{i,t}} = \hat{y}_{i,t} - \mathbf{1}_{i,y_t}$.

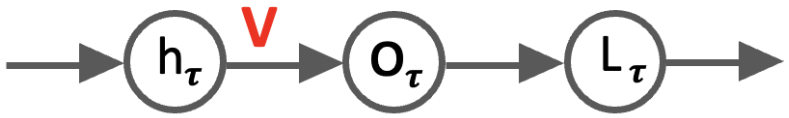


Figure: RNN at the last time step τ does not transfer the hidden state h_τ any further.

- Computing the gradient of the total loss function L at the final time step τ with respect to $\mathbf{h}_\tau$ using the chain rule $\frac{\partial L}{\partial \mathbf{h}_\tau} = \frac{\partial L}{\partial \mathbf{o}_\tau} \frac{\partial \mathbf{o}_\tau}{\partial \mathbf{h}_\tau}$.
- Computing the derivative of $\mathbf{o}_\tau = \mathbf{c} + \mathbf{V}\ \mathbf{h}_\tau \implies \frac{\partial \mathbf{o}_\tau}{\partial \mathbf{h}_\tau} = \mathbf{V}^\mathbf{T}$.
- $\frac{\partial L}{\partial \mathbf{h}_\tau} = \mathbf{V}^\mathbf{T} \frac{\partial \mathbf{L}}{\partial \mathbf{o}_\tau} \implies \nabla_{\mathbf{h}_\tau} \mathbf{L} = \mathbf{V}^\mathbf{T} \nabla_{\mathbf{o}_\tau} \mathbf{L}$.

RNN

Back Propagation Through Time (BPTT)

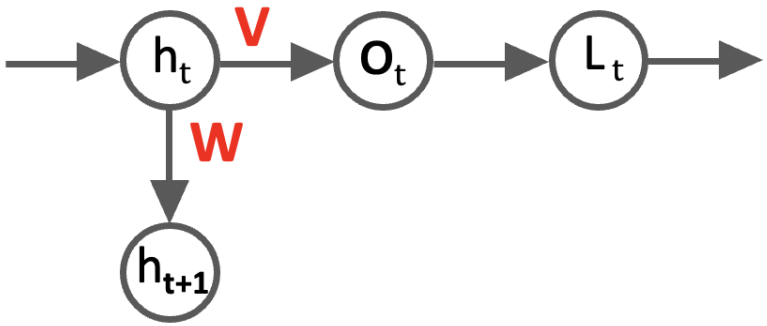


Figure: RNN at the time step t creates an output $\mathbf{o_t}$and transfers the hidden state h_t to the next time step, $t+1$.

- Hidden state $\mathbf{h_t}$ is transmitted to the output $\mathbf{o_t}$ according to $\mathbf{o}_t = \mathbf{c} + \mathbf{V}\ \mathbf{h}_t$, and to the hidden state $\mathbf{h}_{t+1}$ at the next time step, $\mathbf{h}_{t+1} = tanh\,(\mathbf{b} + \mathbf{W}\ \mathbf{h}_t + \mathbf{U}\ \mathbf{x}_{t+1})$.
- Computing the Jacobian of $\mathbf{h}_{t+1}$, $\frac{\partial \mathbf{h_{t+1}}}{\partial \mathbf{h_t}} = \mathbf{W^T}\ \mathrm{diag}(1 - tanh^2\,(\mathbf{b} + \mathbf{W}\ \mathbf{h}_t + \mathbf{U}\ \mathbf{x}_{t+1})) = \mathbf{W^T}\ \mathrm{diag}(1 - (\mathbf{h_{t+1}})^2)$.

RNN

Back Propagation Through Time (BPTT)

- Computing the gradient of the total loss function L at the time step t with respect to $\mathbf{h}_t$ using the chain rule $\frac{\partial L}{\partial \mathbf{h}_t} = \frac{\partial L}{\partial \mathbf{h_{t+1}}} \frac{\partial \mathbf{h_{t+1}}}{\partial \mathbf{h_t}} + \frac{\partial L}{\partial \mathbf{o_t}} \frac{\partial \mathbf{o_t}}{\partial \mathbf{h_t}}$.
- The gardient $\nabla_{\mathbf{h}_t} L = \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t} (\nabla_{\mathbf{h}_{t+1}} L) + \frac{\partial \mathbf{o}_t}{\partial \mathbf{h}_t} (\nabla_{\mathbf{o}_t} L)$.
- Plugging in the values in the gradient, $\nabla_{\mathbf{h_t}} L = \mathbf{W}^T \text{ diag}(1 - (\mathbf{h}_{t+1})^2)(\nabla_{\mathbf{h}_{t+1}} L) + \mathbf{V^T}(\nabla_{\mathbf{o}_t} L)$.
- Computing the gradient of the total loss function L at the time step t with respect to the weights matrix $\mathbf{V}$ using the chain rule $\frac{\partial L}{\partial \mathbf{V}} = \frac{\partial L}{\partial \mathbf{o}} \frac{\partial \mathbf{o}}{\partial \mathbf{V}}$.
- The gradient is computed over all the time steps t, $\frac{\partial L}{\partial \mathbf{V}} = \sum_t \frac{\partial L}{\partial \mathbf{o}_t} \frac{\partial \mathbf{o}_t}{\partial \mathbf{V_t}}$.
- Computing the derivative of $\mathbf{o}_t = \mathbf{c} + \mathbf{V}\ \mathbf{h}_t \implies \frac{\partial \mathbf{o}_t}{\partial \mathbf{V}_t} = \mathbf{h}_t^T$.
- $\frac{\partial L}{\partial \mathbf{V}} = \sum_t \frac{\partial L}{\partial \mathbf{o}_t} \frac{\partial \mathbf{o}_t}{\partial \mathbf{V}_t} \implies \nabla_{\mathbf{V}} L = \sum_t (\nabla_{\mathbf{o}_t} L)\ \mathbf{h}_t^T$.

RNN

Back Propagation Through Time (BPTT)

- Computing the Jacobian of $\mathbf{h}_t = tanh\left(\mathbf{b} + \mathbf{W}\ \mathbf{h}_{t-1} + \mathbf{U}\ \mathbf{x}_t\right)$ with respect to the weights matrix $\mathbf{W}$, $\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}} = \text{diag}(1 - tanh^2\left(\mathbf{b} + \mathbf{W}\ \mathbf{h}_{t-1} + \mathbf{U}\ \mathbf{x}_t\right))\ \mathbf{h}_{t-1}^T = \text{diag}(1 - (\mathbf{h}_t)^2)\ \mathbf{h}_{t-1}^T$.
- The gardient $\frac{\partial L}{\partial \mathbf{W}} = \sum_t \frac{\partial L}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_t} \implies \nabla_{\mathbf{W}} L = \sum_t \text{diag}(1 - (\mathbf{h}_t)^2)\ (\nabla_{\mathbf{h}_t} L)\ \mathbf{h}_{t-1}^T$.
- Similarly, $\nabla_{\mathbf{U}} L = \sum_t \text{diag}\left(1 - (\mathbf{h}_t)^2\right)\ (\nabla_{\mathbf{h}_t} L)\ \mathbf{x}_t^T$.
- $\nabla_{\mathbf{c}} L = \sum_t \nabla_{\mathbf{o}_t} L$.
- $\nabla_{\mathbf{b}} L = \sum_t \text{diag}\left(1 - (\mathbf{h}_t)^2\right)\ \nabla_{\mathbf{h}_t} L$.

RNN

Vanishing Gradient and Exploding Gradient

- This problem exists in RNN because of the recursive nature of RNN, $\mathbf{h}_t = tanh\left(\mathbf{b} + \mathbf{W}^T\ \mathbf{h}_{t-1} + \mathbf{U}^T\ \mathbf{x}_t\right)$.
- If a simpler RNN with linear activation function is assumed, then the hidden node is $\mathbf{h}_t = \mathbf{b} + \mathbf{W}^T\ \mathbf{h}_{t-1} + \mathbf{U}^T\ \mathbf{x}_t$.
- When recursion is applied it results in $\mathbf{h}_t = \mathbf{b} + (\mathbf{W}^t)^T\ \mathbf{h}_0 + \mathbf{U}^T\ \mathbf{x}_t +$ **other terms**.
- It can be further simplified to $\mathbf{h}_t \approx (\mathbf{W}^t)^T\ \mathbf{h}_0$.
- With Singular Value Decomposition the Weight matrix $\mathbf{W}$ is decomposed into Eigen values and Eigen vectors, $\mathbf{h}_t \approx \mathbf{Q}^T \mathbf{\Lambda}\ \mathbf{Q}\ \mathbf{h}_0$.
- Because $\mathbf{\Lambda}$ is a diagonal matrix, the Eigenvalues less than one will become negligibly small, and the values larger than one will explode when $\mathbf{\Lambda}$ is raised to a sufficiently large power t.
- As a result, only the components of $\mathbf{h}_0$ corresponding to the largest Eigenvectors in $\mathbf{Q}$ will survive.

RNN

Skip Connections

- The challenge of vanishing or exploding gradients impacting the retention of past information is overcome by skip connections, or leaky units.
- The idea behind the skip connections is to add a delay d such that the past becomes available after $t + d$ in addition to $t + 1$ from t.
- This creates two paths through time one is an immediate past path and the other is a d-delayed past path.
- One path is finer, and the other is a coarse path through time.
- As a result, the exponential decay τ is reduced to $\frac{\tau}{d}$.
- This allows for overcoming longer-term time dependencies.

RNN

Leaky Units

- Yet another way is to let the hidden state flow in time as is. Replace the weights with one, or near one value.
- $h(t) \leftarrow \alpha h(t-1) + (1-\alpha)v(t)$.
- The linear self-loop allows a smoother and better method to let the hidden states flow in time.
- The values of α can be made free parameters to be tuned in the model, so it would be adjusted over time or can be manually chosen.

Long Short Term Memory (LSTM)

A Practical Implementation of RNN

- LSTM uses a gated approach over the leaky units with simply weighted connections. Hence, it belongs to the category called Gated RNN.
- LSTM adjusts memory transmission at each step compared to leaky units.
- Leaky units lack a mechanism to actively forget distant past when it is no longer relevant, the gates make it possible to actively clear the memory at each step.
- Gated RNNs allow neural networks to learn when to clear the memory.
- LSTM recurrent network is comprised of LSTM memory cells such that there is a loop the same as RNN, but then there is an additional loop that comes from the memory cell.

LSTM

Definitions

- At time t the input to the *j-th* memory cell is x_t^j.
- In the *j-th* memory cell, the memory at time t is c_t^j and is referred as the cell state at that time.
- The cell state is a sort of long-term memory that remembers the information from previous cell states, and it is also incremented by the new information derived from input at that time step.
- h_t^j is the hidden state at time t.

LSTM

- In the *j-th* memory cell, the memory is updated as $c_t^j = f_t^j c_{t-1}^j + i_t^j \tilde{c}_t^j$, where f_t^j is the forget gate, and i_t^j is the input gate.
- c_{t-1}^j is the old memory, and $\tilde{c}_t^j$ is the buffered new memory.
- The amount of old memory replaced is modulated by the forget gate $f_t^j = \sigma(U_f x_t^j + W_f h_{t-1}^j + V_f c_{t-1}^j)^j$.
- Similarly, the amount of new buffered memory added is modulated by the input gate $i_t^j = \sigma(U_i x_t^j + W_i h_{t-1}^j + V_i c_{t-1}^j)^j$.
- The new buffered memory content is $\tilde{c}_t^j = tanh(U_c x_t^j + W_c h_{t-1}^j)$.

LSTM

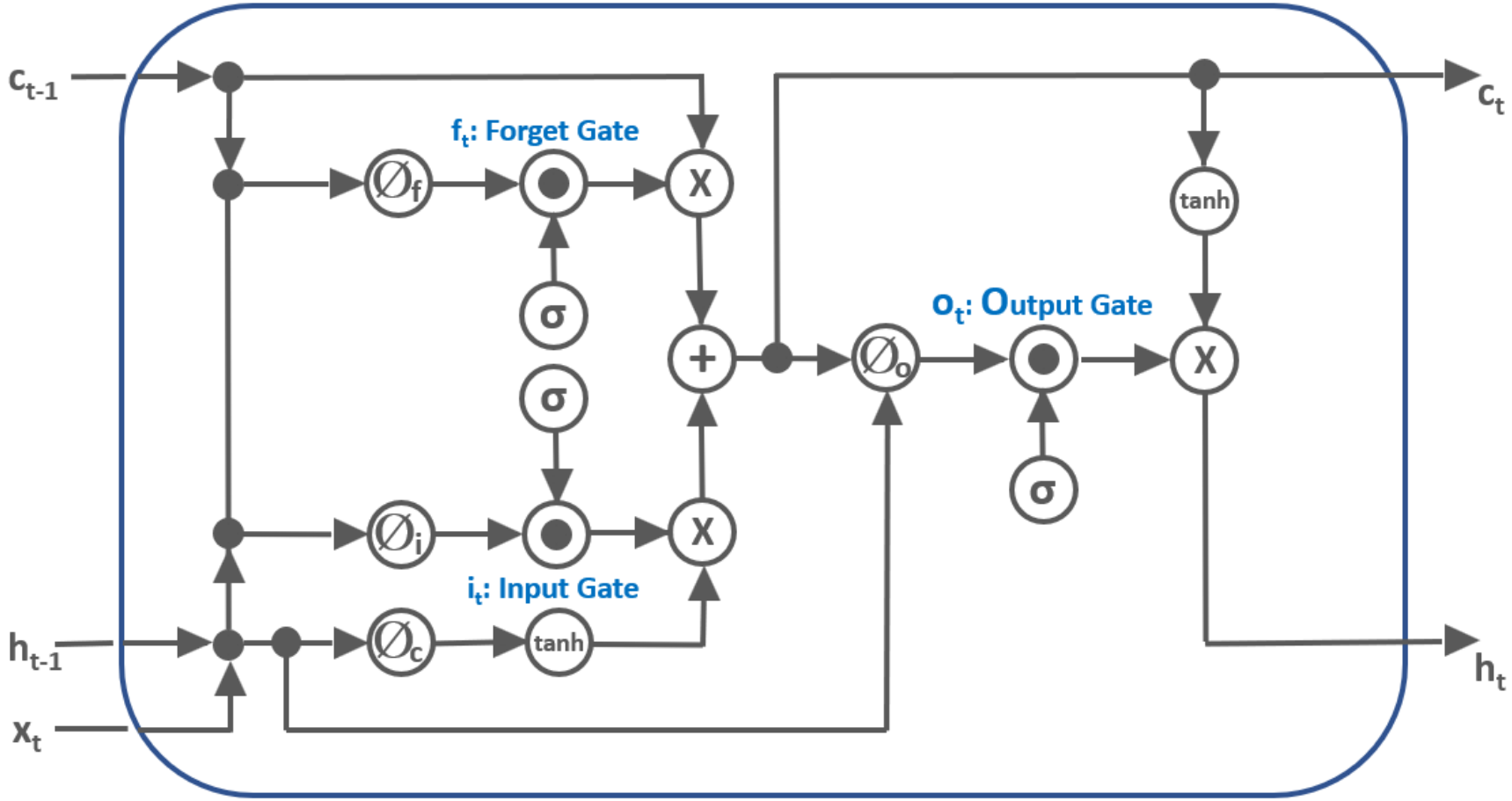


Figure: The *j-th* memory cell shown here with $\varnothing_i = [U_i W_i V_i]$, $\varnothing_f = [U_f W_f V_f]$, $\varnothing_c = [U_c W_c]$, and $\varnothing_o = [U_o W_o V_o]$. For simplicity, the superscript 'j' is omitted. Both outputs of the memory cell c_t and h_t loop back in time. x_t and h_t are also fed into the other memory cells.

LSTM

- The output of the *j-th* memory cell is $h_t^j = o_t^j tanh(c_t^j)$.
- Where $o_t^j = \sigma(U_o x_t^j + W_o h_{t-1}^j + V_o c_t^j)$ is the output gate that modulates the memory content passed to the next LSTM memory cell, and the next step in time.
- If there are three memory cells in each time step, then $j \epsilon \{1, 2, 3\}$.

LSTM

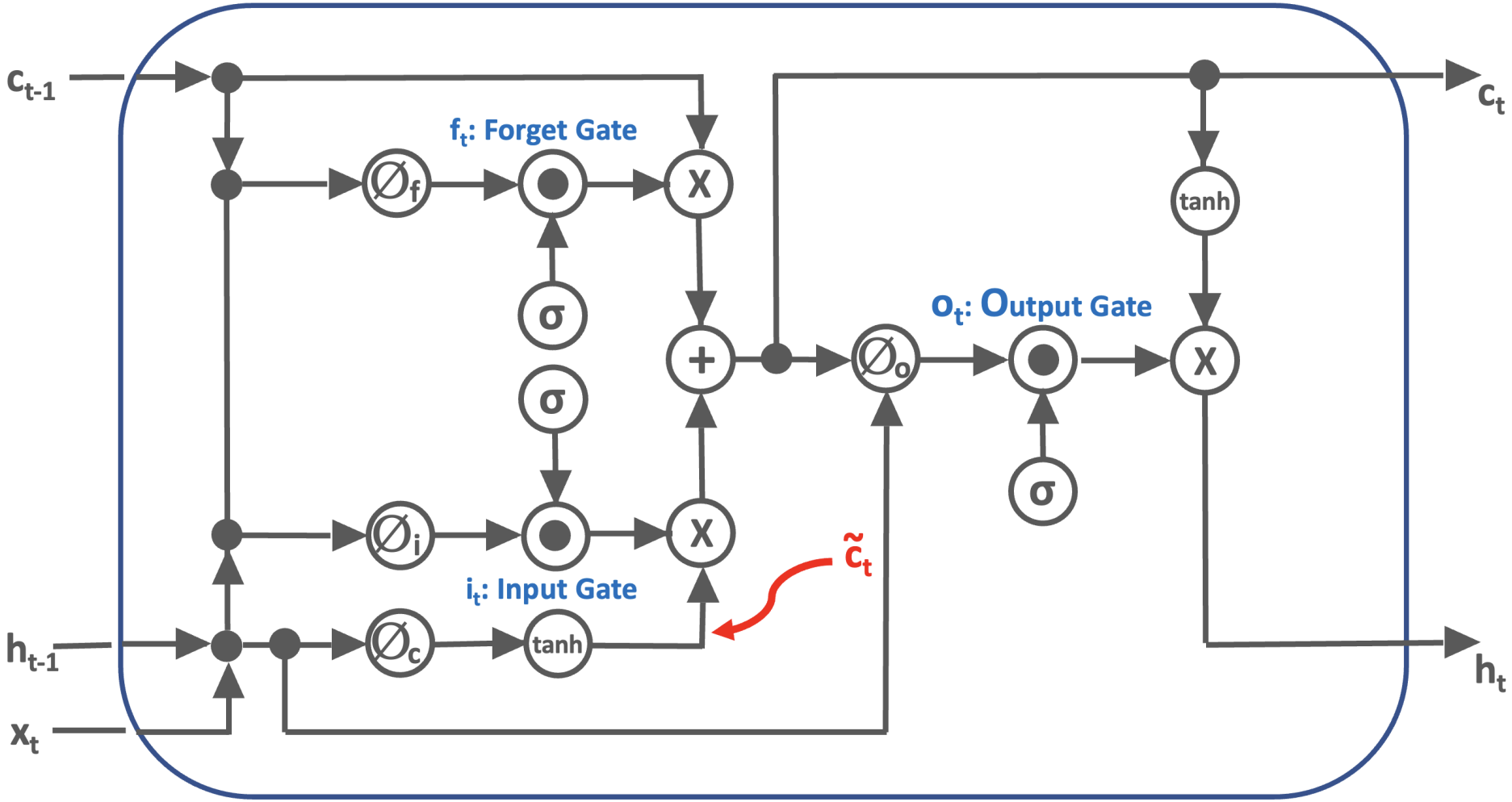


Figure: The new buffered state $\tilde{c}_t^j = tanh(U_c x_t^j + W_c h_{t-1}^j)$ at time t.

LSTM

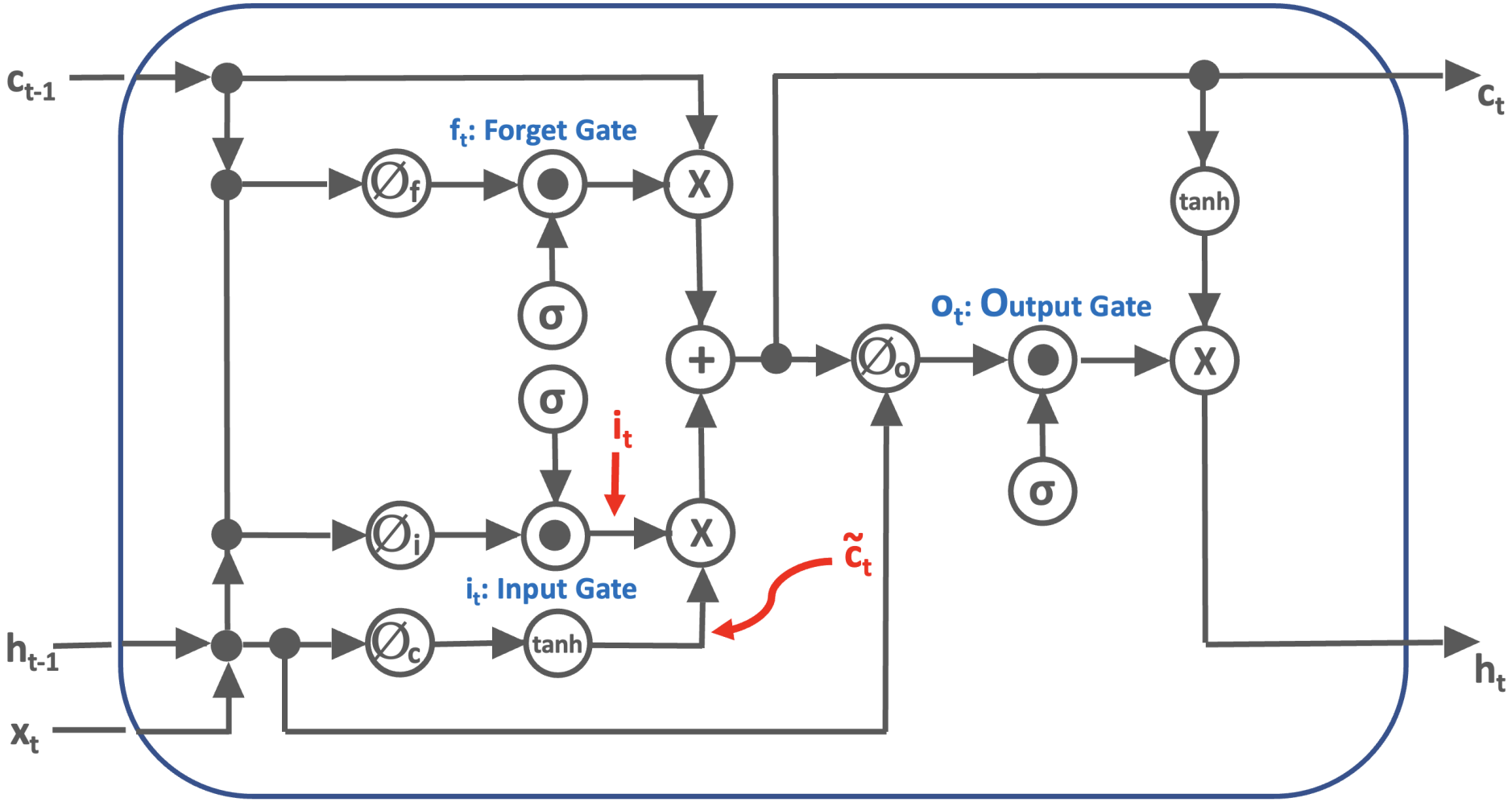


Figure: The input gate $i_t^j = \sigma(U_i x_t^j + W_i h_{t-1}^j + V_i c_{t-1}^j)^j$.

LSTM

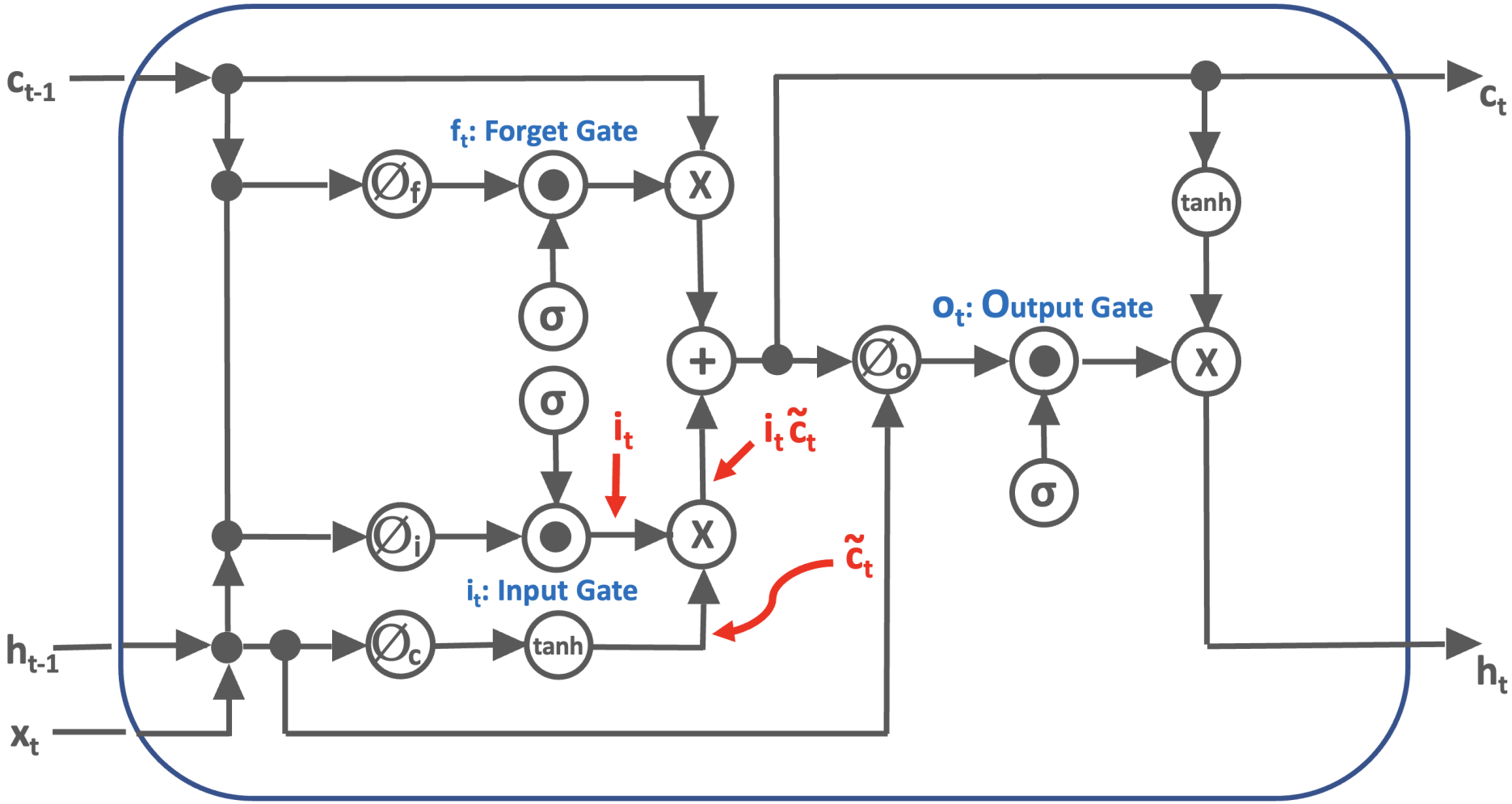


Figure: The gated buffered state $i_t^j \ \tilde{c}_t^j$.

LSTM

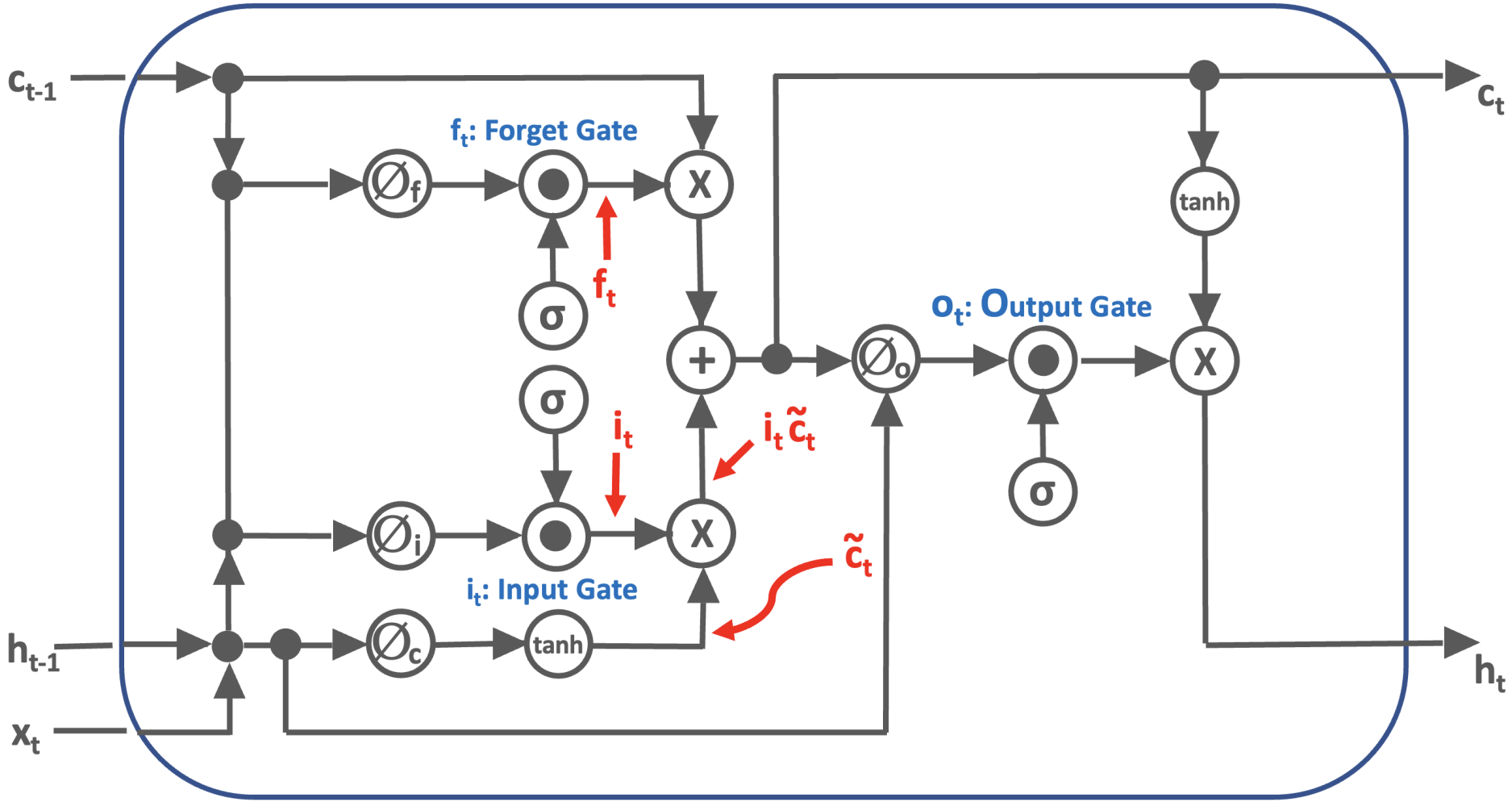


Figure: The forget gate $f_t^j = \sigma(U_f x_t^j + W_f h_{t-1}^j + V_f c_{t-1}^j)^j$.

LSTM

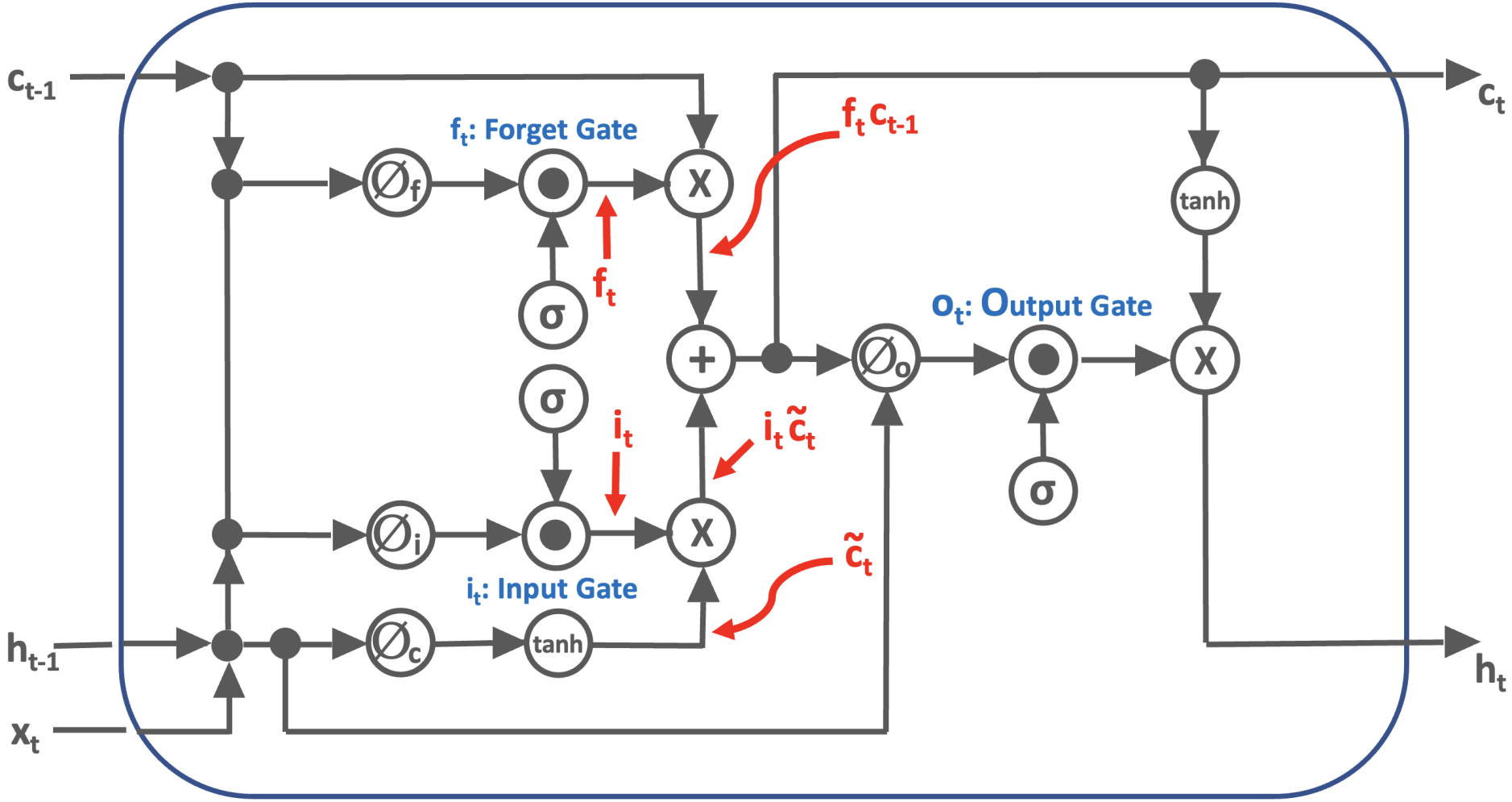


Figure: The gated previous cell state $f_t^j \; c_{t-1}^j$.

LSTM

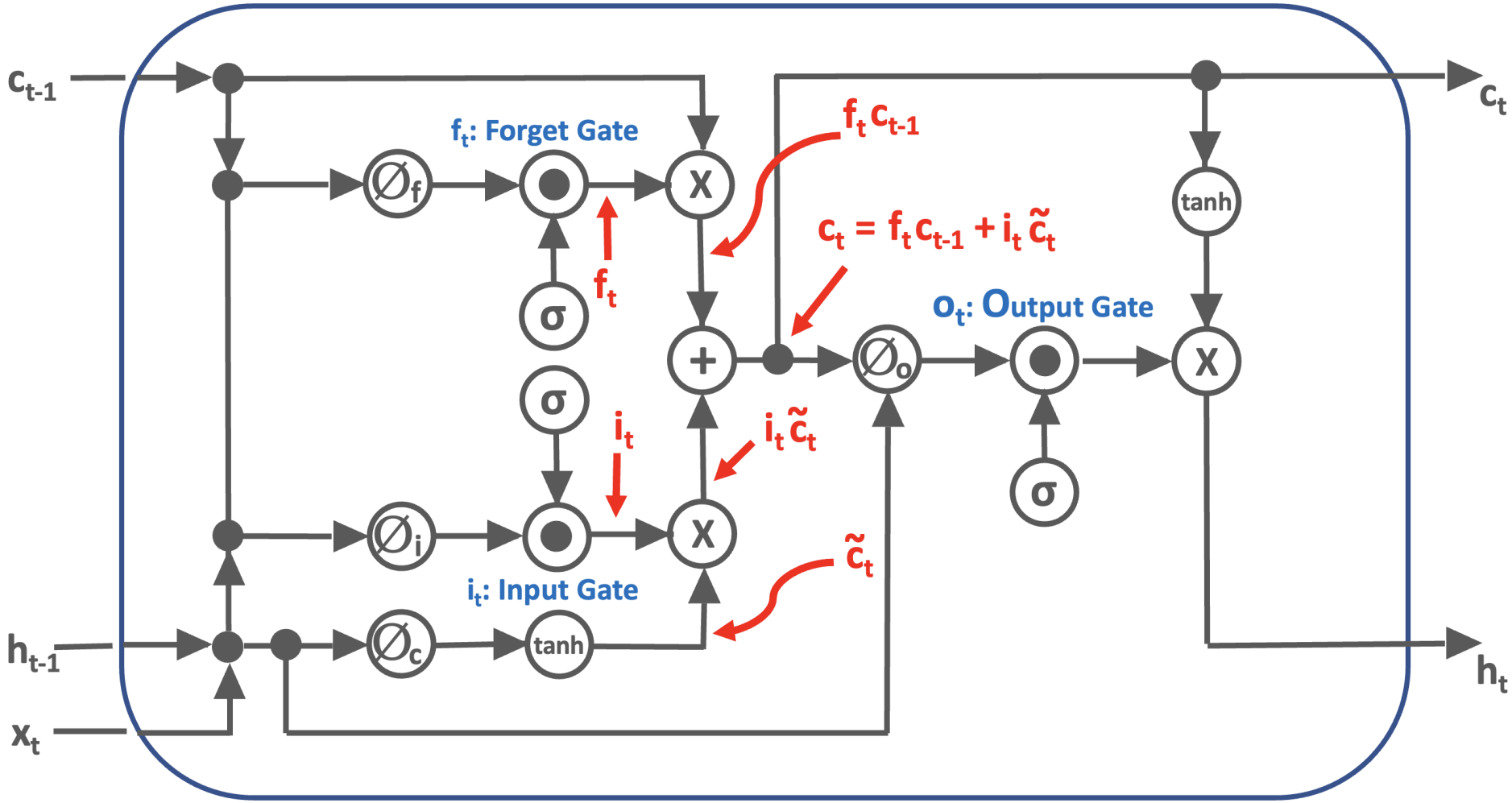


Figure: The cell state $c_t^j = f_t^j c_{t-1}^j + i_t^j \tilde{c}_t^j$ is a combination of long-term and incremental memories.

LSTM

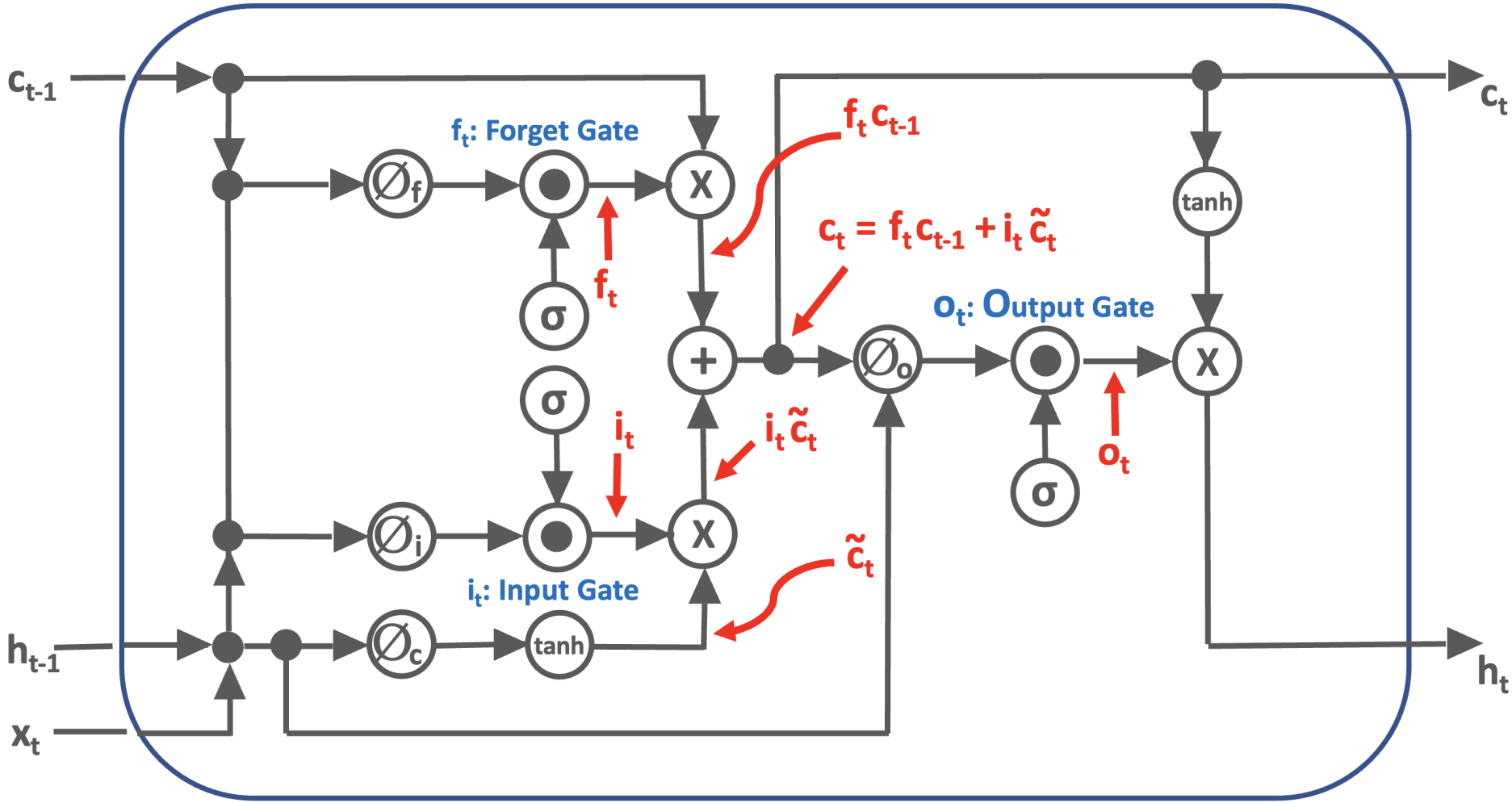


Figure: The output gate $o_t^j = \sigma(U_o x_t^j + W_o h_{t-1}^j + V_o c_t^j)$.

LSTM

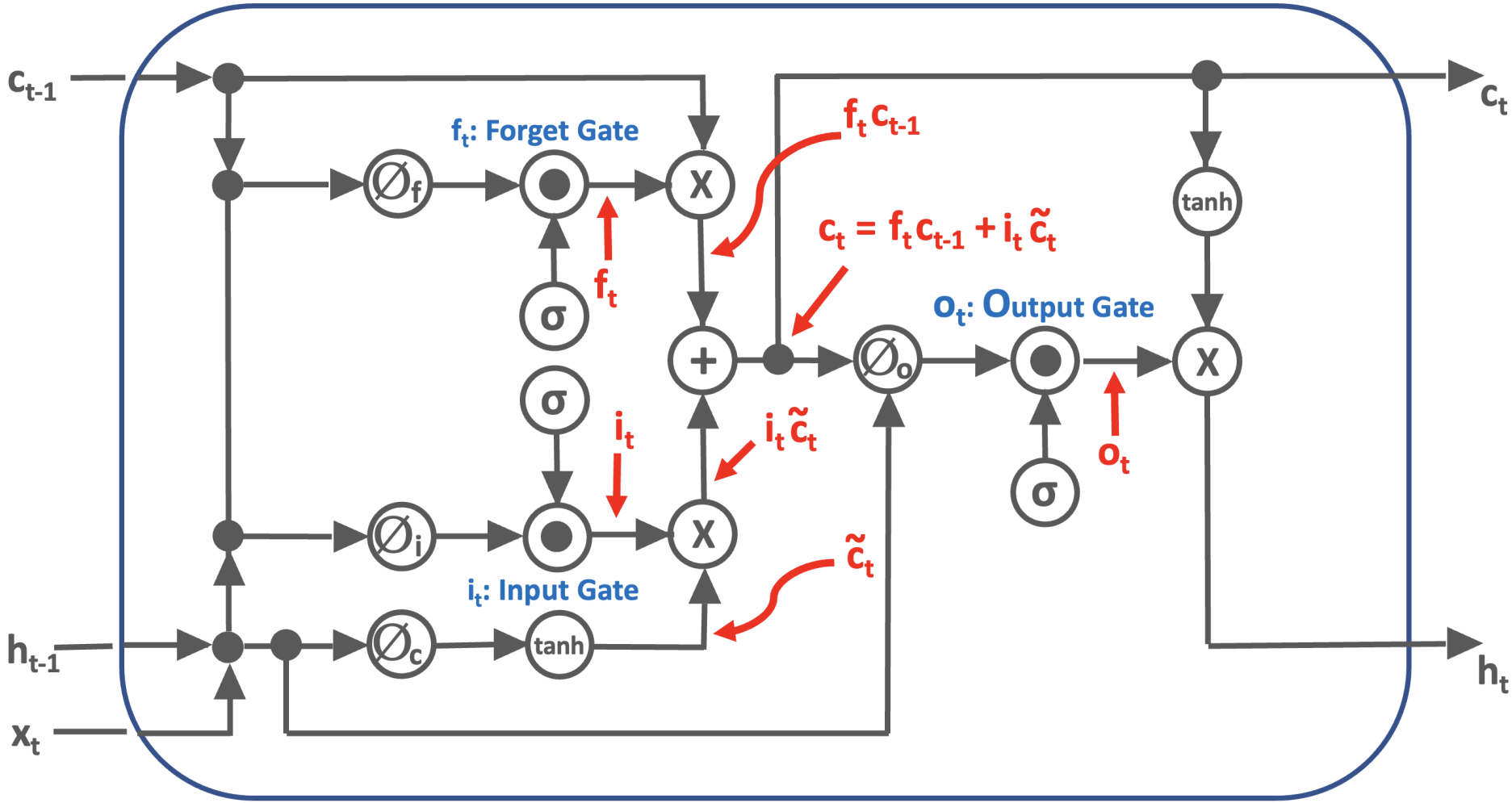


Figure: The new hidden state $h_t^j = o_t^j tanh(c_t^j)$ is a leaked current cell state.